

INSTITUTE CO}ELINE

Introduction to AI and Tools

DAY 1 — PRACTICAL TASKS WORKBOOK

Session: 4 Hours | 12 Tasks | Individual & Pair Exercises

Complete each task using the AI tool of your choice. Write your prompts and observations in the spaces provided. Tasks alternate between general business scenarios and software development contexts.

Participant Name:		Date:	9 September 2026
Job Title:		Group:	

■ Time Block	Section	Tasks	Focus Area
0:00 – 0:40	A — AI Understanding	1–2	AI concepts, tools overview, responsible use
0:40 – 1:30	B — Prompt Engineering	3–5	CRTFC framework, prompt quality, advanced techniques
1:30 – 2:30	C — Communication & Docs	6–8	Emails, documentation, meeting summaries
2:30 – 3:20	D — Data & Code Tasks	9–10	Document analysis, code-related AI tasks
3:20 – 4:00	E — Workflow Design	11–12	Automation templates, personal AI workflow plan

■ Blue tasks = General business context

■■■■ Purple tasks = Software development context

■ Mixed = Both perspectives combined

Completed answers in simple English. Ratings refer to the outputs in this copy. Sample work details are fictional. Add your name, job title and group before submitting.

SECTION A | AI Concepts, Tools & Responsible Use ■ 0:00 – 0:40

Complete Tasks 1–2 in this section. Start from memory, then verify using your AI tool. The goal is to build a solid conceptual foundation before applying AI to real tasks.

Task 1 AI Knowledge Audit & Tool Exploration

■ 20 min

● Beginner

■ Instructions:

1. Without using AI, write your own definition of AI and list what you think it can and cannot do (use the table below).
2. Now open ChatGPT or Claude. Ask: 'Explain what a Large Language Model is in simple terms for a software developer who has never used AI tools before.'
3. Compare the AI's explanation to yours. Note 1 concept that was new or surprising to you.
4. Try a second tool (e.g., if you used ChatGPT, try Claude or Gemini with the exact same prompt). Note any differences in the responses.
5. Ask one of the tools: 'What are the top 5 use cases for AI tools in software development?' Note its answer in the space provided.

■ **Deliverable:** Completed tables + 2 tool comparison notes + developer use cases listed.

My Definition of AI (before AI tool)

AI is software that learns patterns from data. It can help people write, solve problems and make decisions.

AI's Explanation (from the tool)

An LLM learns patterns from lots of text and predicts the next token. It can explain code, but it can also be wrong.

■ AI CAN (your view)

1. Write emails and summaries.
2. Explain code and suggest fixes.
3. Find patterns in data.
4. Draft tests and documentation.

■ AI CANNOT / Risks (your view)

1. Guarantee every answer is correct.
2. Know private facts I have not shared.
3. Make final decisions for me.
4. Keep shared secrets safe by default.

■ *Difference I noticed between Tool 1 and Tool 2:*

Tool 1 - OpenAI assistant: The answer was shorter and explained next-token prediction in plain English.

Tool 2 - Gemini: The answer used a function example and added weights, temperature and the context window.

New concept: a model can sound confident even when its answer is wrong.

■ *Top 5 AI use cases for software development (from AI's answer):*

1. Explain unfamiliar code.
2. Help debug errors.
3. Generate unit tests.
4. Draft documentation.
5. Review and improve code.

Same prompt used for the comparison: Explain what a Large Language Model is in simple terms for a software developer who has never used AI tools before.

Comparison: The first answer was easier to read quickly. Gemini gave more technical detail using familiar programming examples. Both explained text prediction and warned about wrong answers. Gemini was tested live on 9 September 2026.

Task 2 Responsible AI — Risk Scenarios for Developers

■ 20 min

● Beginner

■ Instructions:

1. Read each scenario. Mark SAFE ■ or RISKY ■■ and write a one-sentence reason.
2. For the RISKY ones, write an alternative approach that achieves the same goal safely.
3. After completing your assessment, ask ChatGPT: 'What are the top 5 AI data privacy risks for software developers?' Compare its answer to your assessment.

■ **Deliverable:** Completed risk table + alternative safe approaches written.

Scenario	Safe / Risky?	Reason / Safe Alternative
You paste a function from your company's payment processing module to get a code review	RISKY	It may expose private payment logic. Use an approved tool and a small, sanitized example.
You ask AI: 'Explain how JWT token refresh logic works' without pasting any code	SAFE	It is a general concept question and does not include private code or data.
You paste your production server's nginx config file including the server IP and domain names	RISKY	It reveals infrastructure details. Replace real addresses and share only the relevant settings.
You describe a bug scenario: 'I have a function that processes user objects — it crashes when email is null'	SAFE	It explains the bug without sharing real users or company code.
You upload a CSV of customer order history (with names, emails, and order amounts) to ask for analysis	RISKY	It exposes customer data. Use fake records or anonymous totals in an approved tool.
You ask: 'Write a Python script to parse CSV files with columns: order_id, amount, status'	SAFE	It only shares column names. Use sample data and review the generated script.
You paste your AWS .env file to ask AI why your connection string isn't working	RISKY	The file may contain passwords and keys. Replace secrets with placeholders; rotate any exposed keys.

You ask AI to suggest database index strategies for a table with 10 million rows

SAFE

A generic table-size question shares no private records. Keep the schema anonymous.

Task 2 - AI privacy risks and comparison

Prompt: What are the top 5 AI data privacy risks for software developers?

1. Sharing passwords, API keys or connection strings.
2. Uploading customer names, emails or other personal data.
3. Exposing private source code and internal business logic.
4. Data being stored or used in ways the team has not approved.
5. Connected tools or shared chats giving data to the wrong people.

Comparison: My table covered secrets, customer data and internal code. I also need to check data retention and who can access the tool. I should use approved tools and share only the minimum information needed.

SECTION B | Prompt Engineering Practice ■ 0:40 – 1:30

Tasks 3–5 build your prompt writing skills from basic to advanced. Use the CRTFC framework. Test every prompt in your AI tool and refine based on results.

Task 3 Spot and Fix Weak Prompts

■ 15 min

● Beginner

■ Instructions:

1. Each prompt below is weak. Identify WHY it is weak, then rewrite it using the CRTFC framework.
2. Test your improved version in the AI tool. Rate whether it gave a significantly better result.

■ **Deliverable:** 3 improved prompts written + result quality ratings.

■ Weak Prompt 1 (General): "Write documentation"

Why is it weak? It does not say what to document or who will read it.

CONTEXT	Our team has a new leave-request process for staff.
ROLE	Act as a clear business writer.
TASK	Write instructions: submit request, manager reviews, HR records leave.
FORMAT	Use a short title and numbered steps.
CONSTRAINTS	Use simple English, under 150 words; do not invent policy rules.

Result quality: Much better - clearer and easier to use.

■■■ Weak Prompt 2 (Dev): "Help me with my code"

Why is it weak? No code, error or expected result is provided.

CONTEXT	Python code: <code>def add(a, b): return str(a) + str(b)</code> . <code>add(2, 3)</code> gives "23".
ROLE	Act as a patient Python tutor.
TASK	Make it return the numeric sum and explain the problem.
FORMAT	Show corrected code and a two-sentence explanation.
CONSTRAINTS	Use Python 3, no libraries; assume numeric inputs.

Result quality: Much better - clearer and easier to use.

■ Weak Prompt 3 (General): "Make a summary"

Why is it weak? It gives no source, audience or required length.

CONTEXT	Use the Sprint 14 report in Task 9 for a team update.
---------	---

ROLE	Act as a project assistant.
TASK	Summarize progress, blockers and next actions.
FORMAT	Exactly five bullets, maximum 15 words each.
CONSTRAINTS	Use only the report; do not invent dates or progress.

Result quality: Much better - clearer and easier to use.

■■■ Weak Prompt 4 (Dev): *"Fix the bug"*

Why is it weak? The bug and correct behaviour are not described.

CONTEXT	Task 10 code subtracts 0.10 from 100 for SAVE10 instead of returning 90.
ROLE	Act as a Python developer.
TASK	Fix the percentage discount and explain the change.
FORMAT	Show the corrected function and one input/output example.
CONSTRAINTS	Keep the function name and promo rates; use Python 3.

Result quality: Much better - clearer and easier to use.

■ Weak Prompt 5 (General): *"Write an email about the meeting"*

Why is it weak? No audience, meeting details or purpose is given.

CONTEXT	Use the March 4 sprint planning notes in Task 8.
ROLE	Act as a project coordinator.
TASK	Draft a follow-up email with decisions, actions and blockers.
FORMAT	Use a subject line, short paragraphs and action bullets.
CONSTRAINTS	Under 180 words; keep names and dates; flag missing details.

Result quality: Much better - clearer and easier to use.

Quick result checks:

1. Leave guide: submit the request, wait for manager review, then HR records it.
 2. add(2, 3) now returns 5 using return a + b.
 3. Five-bullet summary: see Task 9, page 13.
 4. SAVE10 on 100 now returns 90: see the Task 10 results.
 5. Meeting email: launch moves to March 17; Nour informs the client; Khalid updates the team on Friday.
- Observation: Giving the source and expected format makes the answers much easier to use.

Task 4 Build a Level-4 Expert Prompt

■ 20 min

● Intermediate

■ Instructions:

1. Choose ONE scenario below (circle your choice).
2. Scenario A (General): You are a project manager. Write a status email to the steering committee summarizing milestone progress, a risk, and the next sprint goal.
3. Scenario B (Dev): You are a backend developer. Ask AI to review a Python function that processes payment transactions for correctness, security, and edge cases.
4. Scenario C (Dev): You are a tech lead. Ask AI to compare three caching strategies (Redis, Memcached, in-memory) for a high-traffic API with specific tradeoffs.
5. Fill in the CRTFC table below, then write your full prompt in a single paragraph.
6. Test the prompt. Rate the result. Then write one follow-up prompt to refine the output.

■ Deliverable: Full CRTFC prompt + follow-up refinement prompt + quality rating.

I chose Scenario: A (sample internal project).

CONTEXT	I manage an internal portal project. Design is done; API work is 80% complete.
ROLE	Act as a project manager writing to the steering committee.
TASK	Draft a status email covering milestones, QA risk and the next sprint goal.
FORMAT	Subject line plus three short sections; professional, simple English.
CONSTRAINTS	Under 150 words. QA capacity is limited. Next goal: finish API and start testing.

■ My Full Prompt (combine all CRTFC elements):

Act as a project manager. I manage an internal portal project: design is complete and API work is 80% done. Write a status email to the steering committee with milestones, the risk of limited QA capacity, and the next sprint goal of finishing the API and starting testing. Use a subject line and three short sections. Keep it professional, simple and under 150 words. Do not invent dates or progress.

■ Follow-up refinement prompt (what did you ask next to improve the output?):

Make the email shorter and add a clear request for one QA engineer for the next sprint. Keep the same facts and use a polite tone.

Refinement: the email now says what support is needed. The refined request is clear and polite.

Initial result quality: Good. After refinement: Excellent.

Task 5 Advanced Prompting Techniques

■ 15 min

● Advanced

■ Instructions:

1. Practice each advanced technique using the instructions below.
2. **TECHNIQUE A** — Chain of Thought: Ask AI: 'Explain step by step how a JWT token is validated on an API server. Then identify 3 potential security vulnerabilities in a basic implementation.' Paste the response summary below.
3. **TECHNIQUE B** — Few-Shot Prompting: Write a prompt that gives the AI 2 examples of a code comment format you want, then asks it to apply that format to a new function.
4. **TECHNIQUE C** — Iterative Refinement: Start with a simple prompt, then write 2 follow-up prompts that progressively refine the output (e.g., add error handling, then add logging, then add a unit test).

■ **Deliverable:** 3 technique examples attempted + observations noted.

Technique A — Chain of Thought

Prompt: Explain the main steps of JWT validation on an API server, then list three security risks.

Summary: Read the bearer token; use a trusted library; allow only expected algorithms; verify the signature with the trusted key; check expiry, issuer, audience and other required claims; then check permissions.

Risks: accepting an unsigned token, skipping expiry or audience checks, and leaking tokens in logs.

Observation: a valid token still needs permission checks.

Technique B — Few-Shot Prompting

Prompt: Follow this comment format. Example 1: `def add(a, b): return a + b -> # Return the sum of two numbers.` Example 2: `def is_even(n): return n % 2 == 0 -> # Check whether a number is even.` Now add one matching comment to `def square(n): return n * n.` Return only the comment.

Output: `# Return the square of a number.`

Observation: the examples made the comment short and consistent.

Technique C — Iterative Refinement (show all 3 prompts)

1. Write a Python function that returns the average of a list of numbers.
2. Refine it: reject None and non-numeric values with `ValueError`, and return 0 for an empty list.
3. Refine it again: add `pytest` tests for normal input, empty input, None, invalid values and zero.

Observation: each prompt adds a clear requirement, so the final version is easier to check.

SECTION C | AI for Communication & Documentation ■ 1:30 – 2:30

Tasks 6–8 apply AI to real communication and documentation scenarios. All prompts should use the CRTFC framework. Save your best prompts — you will reuse them in Section E.

Task 6 Professional Email Writing — General & Developer Scenarios

■ 20 min

● Intermediate

■ Instructions:

1. Write and test a professional email for BOTH scenarios below (one general, one developer-specific).
2. SCENARIO 1 (General): You are a product manager. Write an email to senior management requesting approval for a 2-week delay in the product launch due to critical QA findings. Keep it under 200 words, professional, and include a revised timeline.
3. SCENARIO 2 (Dev): You are a backend developer. Write an email to the DevOps team requesting emergency access to production logs to investigate a P1 incident impacting 15% of users. Include: the affected service, symptoms, and what you need access to.
4. For each: fill in the CRTFC table, write the full prompt, test it, and rate the result.
5. BONUS: Ask AI to rewrite your best email in a different tone (more urgent / more formal / more concise).

■ **Deliverable:** 2 complete email prompts + both emails reviewed and rated.

■ Scenario 1: General — Launch Delay Approval

CONTEXT	Sample launch: 10 March to 24 March; critical QA bugs need two more weeks.
ROLE	Act as a product manager writing to senior management.
TASK	Request approval for a two-week delay and give the revised timeline.
FORMAT	Subject, greeting, short body and sign-off; professional tone.
CONSTRAINTS	Under 200 words. Fix 11-17 March, retest 18-23 March, launch 24 March.

Full prompt:

Prompt: Act as a product manager. Request approval to delay a sample launch from 10 to 24 March because of critical QA bugs. Include the revised plan; be professional and stay under 200 words.

Email - Subject: Approval for launch delay

Dear Management Team,

Please approve moving our launch from 10 to 24 March due to critical QA findings. We will fix the issues on 11-17 March and retest on 18-23 March. We will launch on 24 March if the critical issues are resolved.

Regards, Product Manager

Review: Ready to send for this sample scenario.

■■■ Scenario 2: Dev — Emergency Production Access

CONTEXT	Sample P1 incident: order API returns errors for 15% of users.
---------	--

ROLE	Act as a backend developer writing to the DevOps team.
TASK	Request emergency read-only access to the affected service logs.
FORMAT	Subject, short body, affected service, symptoms and access needed.
CONSTRAINTS	Request approved two-hour access; mask personal data; never ask for credentials.

Full prompt:

Prompt: Act as a backend developer. Email DevOps for urgent, approved read-only log access: a P1 order API issue affects 15% of users. Include symptoms, logs needed and access duration.

Email - Subject: P1 order API log access

Hi DevOps Team,

The order API is returning server errors for 15% of users. Please approve two hours of read-only access to application and error logs from the last 60 minutes, with personal data masked. Please confirm urgently so I can investigate.

Thanks, Backend Developer

Review: Ready to send for this sample scenario.

■ **BONUS** — *Tone Variation (paste the AI's alternative version below):*

Subject: Urgent: P1 order API log access

Hi DevOps team,

A P1 issue in the order API is causing server errors for 15% of users. Please approve two hours of read-only access to the last 60 minutes of application and error logs, with personal data masked. Please confirm access as soon as possible so I can investigate.

Thanks, Backend Developer

Task 7 Technical Documentation Generation

■ 25 min

● Intermediate

■ Instructions:

1. Practice generating three types of technical documentation using AI.
2. DOC TYPE 1 — Docstring: Write a prompt asking AI to add a Google-style docstring to the sample Python function provided below.
3. DOC TYPE 2 — README Section: Write a prompt to generate an 'Installation & Setup' README section for a Node.js REST API. Include: prerequisites, clone, install dependencies, configure .env, run dev server.
4. DOC TYPE 3 — Commit Message: Write a prompt asking AI to generate a conventional commit message for these changes: 'Added Redis caching layer to the /products endpoint, reducing average response time from 480ms to 35ms. Also fixed a null pointer bug when product category is undefined.'
5. Test all three prompts. Note which output required the least editing.

■ **Deliverable:** 3 documentation prompts written + one 'least editing needed' observation.

```
# Sample function for documentation exercise:
def process_user_order(user_id, items, discount_code=None):
    total = sum(item['price'] * item['qty'] for item in items)
    if discount_code:
        discount = get_discount(discount_code)
        total = total * (1 - discount)
    if total < 0:
        raise ValueError('Total cannot be negative')
    return {'user_id': user_id, 'total': round(total, 2), 'items_count': len(items)}
```

Doc Type 1 — Docstring Prompt:

Context: Use `process_user_order` from this workbook. Role: Act as a Python developer. Task: Add a Google-style docstring. Format: Summary, Args, Returns and Raises. Constraints: Keep the code unchanged; do not guess the `user_id` type or what `get_discount` does. Explain that `items_count` counts item entries, not quantities.

Observation: The docstring explains inputs, the returned dictionary and possible errors.

Doc Type 2 — README Section Prompt:

Context: A sample Node.js REST API. Role: Act as a technical writer. Task: Write Installation & Setup. Format: Numbered steps and command blocks. Constraints: Include Node/npm prerequisites, clone, dependency install, `.env` configuration and dev start. Use `[REPO_URL]` and `[PROJECT_DIR]`. Mark `.env.example` and `npm run dev` as assumptions to check in the repo. Never include real secrets.

Observation: The setup steps are clear; the project-specific settings still need checking.

Doc Type 3 — Commit Message Prompt:

Context: Redis caching was added to `/products` (480ms to 35ms), and an undefined-category bug was fixed. Role: Act as a developer. Task: Write a conventional commit. Format: One subject and a short body. Constraints: Keep the supplied facts and do not invent a breaking change.

Output: `feat(products): add Redis caching`

Reduce average response time from 480ms to 35ms.

Fix the null pointer bug when product category is undefined.

Least editing needed: Commit Message.

Why? The changes were clear, so the short result covered both points.

Task 8 Meeting Notes & Report Generation

■ 15 min

● Intermediate

■ Instructions:

1. Read the raw meeting notes below.
2. Write a prompt asking AI to organize these notes into a professional meeting summary with sections: Attendees, Key Decisions, Action Items (with owner and deadline), and Blockers.
3. Write a second prompt asking AI to convert the same notes into a 5-slide presentation outline.
4. Compare both outputs and note which format is most useful for different audiences.

■ **Deliverable:** 2 prompts written + audience suitability comparison noted.

MEETING NOTES – Sprint Planning – March 4 Present: Ahmed (backend), Sara (frontend), Khalid (DevOps), Nour (PM) Ahmed - auth service refactor done, PR open, needs review by EOD tomorrow Sara - dashboard blocked waiting for API spec from Ahmed Khalid - deployment pipeline broken on staging, no ETA yet, investigating Agreed - launch date pushed from March 10 to March 17 because of pipeline issue Nour - needs to update client on delay - email by tomorrow morning Ahmed to review Sara's frontend PR before end of week Khalid to give update on pipeline at daily standup Friday Next sprint planning March 11 at 10am

■ Prompt 1 — Meeting Summary:

Context: Use the March 4 sprint notes above. Role: Act as a project coordinator. Task: Organize them into a meeting summary. Format: Attendees, Key Decisions, Action Items with owner and deadline, and Blockers. Constraints: Use only the notes; write "not stated" for missing owners or deadlines. Keep March dates and do not guess the year.

Summary: Ahmed, Sara, Khalid and Nour attended. Launch moves from March 10 to March 17 because staging is broken. Next planning is March 11 at 10am. Sara is blocked on the API spec. Pipeline repair has no ETA. Action table below.

■ Prompt 2 — Presentation Outline:

Context: Use the same March 4 notes. Role: Act as a presentation writer. Task: Create a five-slide outline for management. Format: Five titles with short bullets. Constraints: Keep the facts, do not invent dates, and highlight the impact of blockers.

1. Sprint Planning: March 4; Ahmed, Sara, Khalid and Nour.
2. Progress: auth refactor complete; PR needs review.
3. Blockers: dashboard API spec and broken staging pipeline.
4. Decision and actions: launch March 17; client update and PR reviews.
5. Next steps: Friday pipeline update; planning March 11 at 10am.

Meeting summary: All - useful for tracking owners and deadlines.

Presentation outline: Management / client meetings - useful for a quick overview.

Action items from the meeting

Action	Owner	Deadline
Review auth service PR	Not stated	March 5, end of day
Share dashboard API spec	Ahmed	Not stated
Review Sara's frontend PR	Ahmed	End of week
Email client about delay	Nour	March 5, morning
Give pipeline update	Khalid	Friday standup
Investigate staging pipeline	Khalid	No ETA

SECTION D | AI for Data Analysis & Code Tasks ■ 2:30 – 3:20

Tasks 9–10 focus on two of the highest-value AI use cases for developers: data and document analysis, and code-related tasks including review, debugging, and test generation.

Task 9 Document Analysis & Data Extraction

■ 25 min

● Intermediate

■ Instructions:

1. Use the sample sprint report below for all prompts in this task.
2. PROMPT 1: Write a prompt to summarize the report in exactly 5 bullet points (max 15 words each).
3. PROMPT 2: Write a prompt to extract all action items as a table with columns: Action | Owner | Deadline | Priority.
4. PROMPT 3: Write a prompt to convert the 'Performance Metrics' section into a structured JSON object.
5. PROMPT 4 (Advanced): Write a prompt asking AI to identify risks mentioned in the report and suggest mitigations.
6. Test all 4 prompts. Rate each for output quality and immediate usability.

■ **Deliverable:** 4 prompts written + quality ratings + JSON output from Prompt 3 pasted.

SPRINT 14 PERFORMANCE REPORT – Week of March 3 COMPLETED: - User authentication service refactored (owner: Ahmed, 3 days ahead of schedule) - Payment gateway integration tested – all 47 test cases passing - Mobile app API endpoints v2 deployed to staging IN PROGRESS: - Customer dashboard frontend (Sara, 60% complete, blocked on API spec) - CI/CD pipeline fix for staging environment (Khalid, ETA: March 7) BLOCKERS: - Staging deployment broken since March 2 – no prod deployments possible until resolved - API spec for dashboard not finalized – Sara blocked for 2+ days PERFORMANCE METRICS: - Sprint velocity: 42 story points (target: 45, -6.7%) - Bug count opened this sprint: 7 (4 resolved, 3 open) - Code review turnaround: avg 18 hours (target: 24 hours) ■ - Test coverage: 74% (target: 80%, gap: -6%) ACTION ITEMS: - Ahmed: Share API spec with Sara by March 5 - Khalid: Resolve pipeline issue by March 7 - Nour: Notify client of revised launch date by March 5 - All: Increase test coverage to 80% before next sprint review

Prompt 1 — 5-Bullet Summary:

Prompt: Summarize the Sprint 14 report in exactly five bullets, maximum 15 words each. Use only the report.

- Authentication refactor finished early; all 47 payment tests passed; mobile API v2 reached staging.
- Dashboard is 60% complete and blocked by an unfinished API spec.
- Broken staging pipeline prevents production deployments; Khalid targets March 7.
- Velocity is 42 points; three bugs remain; reviews average 18 hours; coverage is 74%.
- Ahmed and Nour have March 5 actions; everyone must improve coverage before review.

Quality: Ready to use - five bullets, each within 15 words.

Prompt 2 — Action Items Table:

Prompt: Extract every action item from Sprint 14 as Action | Owner | Deadline | Priority. Mark priorities as suggested because the report does not state them.

Action	Owner	Deadline	Priority*
Share API spec	Ahmed	March 5	High
Fix staging pipeline	Khalid	March 7	Critical
Notify client	Nour	March 5	High
Raise coverage to 80%	All	Before sprint review	High

*Priorities are suggested, not stated in the report.

Quality: Ready to use - priorities clearly marked as suggestions.

Prompt 3 — JSON Conversion:

Prompt: Convert only the Performance Metrics section of Sprint 14 into valid JSON. Use descriptive snake_case keys and numeric values. Include actual values, targets, bug counts and units in key names. Express the coverage gap as percentage points. Return JSON only.

Quality check: valid structure and all reported metrics included.

Quality: Ready to use.

Prompt 4 — Risk Analysis:

Prompt: Identify the report's risks and suggest practical mitigations. Separate reported facts from suggestions. Do not invent owners or dates.

Pipeline blocks releases: prioritize the repair and verify staging before deployment.

Missing API spec delays Sara: share the spec and agree the contract.

Coverage is 74%, with three open bugs: add tests and triage remaining bugs.

Velocity is below target: review scope and blockers before committing more work.

Quality: Ready to use.

JSON output from Prompt 3:

```
{"sprint_velocity": {"actual_points": 42, "target_points": 45, "variance_percent": -6.7},  
"bugs": {"opened": 7, "resolved": 4, "open": 3},  
"code_review": {"average_hours": 18, "target_hours": 24},  
"test_coverage": {"actual_percent": 74, "target_percent": 80, "gap_percentage_points": -6}}
```

Task 10

AI-Assisted Code Review, Debugging & Test Generation

■ 25 min

● Advanced

■ Instructions:

1. Complete the 3 code-related sub-tasks below using the sample code provided.
2. SUB-TASK A — Code Review: Write a CRTFC prompt asking AI to review the Python function below for: correctness, edge cases, security, and PEP8 compliance. Then test it.
3. SUB-TASK B — Bug Investigation: The function below has a logic bug. Write a prompt asking AI to identify the bug and provide a fixed version with an explanation.
4. SUB-TASK C — Test Generation: Write a prompt asking AI to generate pytest unit tests for the fixed version of the function, covering: happy path, empty input, null values, and boundary conditions.
5. REFLECTION: Which sub-task produced the most impressive AI output? Note what surprised you.

■ Deliverable: 3 CRTFC prompts + all results noted + reflection written.

```
# Code for review – find the bug and write tests: def calculate_discount(cart_items,
promo_code): """Apply discount to cart total.""" PROMO_CODES = { 'SAVE10': 0.10,
'SAVE20': 0.20, 'VIP50': 0.50 } total = 0 for item in cart_items: total += item['price']
* item['quantity'] if promo_code in PROMO_CODES: discount = PROMO_CODES[promo_code] total
= total - discount # BUG IS HERE if total < 0: total = 0 return total
```

Sub-Task A — Code Review Prompt:

Context: Review calculate_discount from this workbook. Role: Act as a Python code reviewer.

Task: Check correctness, edge cases, security and PEP8. Format: A short issue list with severity, reason and fix. Constraints: Use Python 3, keep the current promo codes and do not invent other code. Treat prices as non-negative numbers and quantities as non-negative integers; reject invalid entries.

Focus on practical problems and include one example of the wrong result.

What did AI find / produce?

High: SAVE10 subtracts 0.10 instead of 10%; 100 becomes 99.90, not 90.

Medium: missing keys, None and invalid numbers can crash or distort totals. Validate inputs. Check promo eligibility on the server in a real app.

Style: use proper indentation and move PROMO_CODES to module level.

Money: use Decimal in production to avoid float rounding issues.

Sub-Task B — Bug Investigation Prompt:

Context: SAVE10 on a total of 100 returns 99.90. Role: Act as a Python developer. Task: Fix the percentage calculation and validate inputs. Format: Corrected function and a short explanation.

Constraints: Keep the function name and rates. Empty cart returns 0; None cart or invalid item values raise ValueError; None or unknown promo means no discount. Reject non-finite or negative prices and non-integer or negative quantities.

What did AI find / produce?

The bug is total - discount. It should be total * (1 - discount).

For example: $100 * (1 - 0.10) = 90$.

The fixed function checks the cart and each item before calculating. Empty carts return 0, and invalid input raises ValueError. Unknown or None promo codes keep the total unchanged.

Check: SAVE20 on 100 gives 80; VIP50 on 100 gives 50.

Sub-Task C — Test Generation Prompt:

Context: Use calculate_discount with the corrected percentage calculation and input checks.

Role: Act as a Python tester. Task: Generate pytest tests for happy paths, empty input, null values and boundary conditions. Format: A runnable test module with parametrized cases.

Constraints: Use pytest.approx for numeric results. Check all three promos, an unknown promo, None promo, zero values, missing keys, negative values and invalid types. No network calls.

Run the tests and report the actual result.

What did AI find / produce?

Generated tests check the original bug, all three discount rates, mixed carts, empty input, zero price and zero quantity. They also check None, missing keys, invalid types, negative values and non-finite prices.

Execution result: 25 tests passed (pytest, 0.27 seconds).

The tests confirm the expected results and error handling.

■ Reflection — Which sub-task surprised you most and why?

Test generation surprised me most. I first thought about the normal discount, but the tests also covered None, missing fields and invalid quantities. This helps me catch problems earlier. I still need to review the tests and make sure the expected behaviour is right.

SECTION E | Automation Templates & Personal AI Workflow ■ 3:20 – 4:00

Tasks 11–12 turn everything you practiced today into a reusable personal system. The templates you build here will save you hours every week.

Task 11 Build 3 Personal Reusable Prompt Templates

■ 25 min

● Intermediate

■ Instructions:

1. Build ONE reusable prompt template for each of the 3 categories below.
2. Use [PLACEHOLDERS] for the parts that change each time. Include all 5 CRTFC elements.
3. TEMPLATE 1 (General): A recurring communication task from your work (weekly update, client email, meeting request, etc.)
4. TEMPLATE 2 (Dev): A recurring development task (code review, PR description, bug report, test generation, documentation)
5. TEMPLATE 3 (Your choice): Any other task you do regularly that AI could help with
6. For each template: write it, test it by filling in the placeholders, then refine it once.

■ **Deliverable:** 3 complete reusable prompt templates with [PLACEHOLDERS].

■ General — Template 1: General Communication Template

Task this template is for: Weekly update

Context: My team's week is [WEEK]; updates are [UPDATES]. Role: Act as a project coordinator. Task: Draft a weekly update for [AUDIENCE]. Format: Progress, blockers and next steps. Constraints: Under [WORD_LIMIT] words; use only supplied facts; mark missing owners as not stated.

Refinement: Added a rule for missing owners.

■■■ Dev — Template 2: Developer Workflow Template

Task this template is for: Code review

Context: Review [LANGUAGE] code: [CODE]; expected behaviour: [EXPECTED]. Role: Act as a code reviewer. Task: Find bugs and edge cases. Format: Issue, reason and suggested fix. Constraints: Prioritize important issues; keep the public API; do not assume missing code.

Refinement: Added expected behaviour and issue priority.

■ Custom — Template 3: My Choice Template

Task this template is for: Study plan

Context: I am learning [TOPIC], with [MINUTES] minutes daily for [DAYS] days. Role: Act as a tutor. Task: Build a beginner plan. Format: Day, topic and practice task. Constraints: Keep within my time limit; use simple English and free resources.

Refinement: Added a daily time limit and one practice task.

Test examples: Sprint 14 update; SAVE10 code review; Python study plan (20 minutes daily for 3 days).

Task 12 Design Your Personal AI Developer Workflow

■ 15 min

● Advanced

■ Instructions:

1. List your top 8 most repetitive or time-consuming development and communication tasks.
2. For each, complete the row in the workflow table: tool, prompt type, and estimated time saved per week.
3. Circle your top 3 highest-impact tasks.
4. Prepare to present ONE workflow example to the group in 60 seconds — explain: what the task is, which AI tool you will use, and how much time you expect to save.

■ Deliverable: Completed workflow table + 1 example prepared for group sharing.

#	Task Description	Type	AI Tool	Template Name	Est. Time Saved/Week
1	Weekly updates	General	ChatGPT	Weekly update	30 min
2	Meeting summaries	General	ChatGPT	Meeting summary	30 min
3	Client emails	General	ChatGPT	Client email	20 min
4	Code review	Dev	ChatGPT	Code review	60 min
5	Unit test drafts	Dev	ChatGPT	Test generation	90 min
6	Bug investigation	Dev	ChatGPT	Bug investigation	60 min
7	README and docstrings	Dev	ChatGPT	Documentation	45 min
8	Commit messages	Dev	ChatGPT	Commit message	15 min

■ My 60-Second Workflow Pitch (prepare here):

The task I chose: Drafting unit tests for Python functions.

AI tool I will use: ChatGPT.

Template: Give the function, expected results and required edge cases.

I review the generated pytest cases and run them before using them.

Estimated time saved: 90 min/week. Impact: more time for fixing bugs.

■ Day 1 — Final Reflection

Complete before the session ends:

The most useful general AI use case I discovered today:

Turning rough meeting notes into clear actions with owners and deadlines.

The most useful developer AI use case I will start this week:

Drafting unit tests, then reviewing and running them myself.

One thing I now understand about AI that I did not before:

AI predicts likely answers, so clear wording does not guarantee accuracy.

One question I still have about using AI in my development workflow:

How can I use company code with AI while following our data rules?

My commitment: the first AI-assisted task I will do at work tomorrow:

I will draft a weekly update from non-sensitive notes and check every detail.

Top three highest-impact tasks: unit test drafts, code review and bug investigation (circled in the table).

Estimated total time saved: 350 minutes per week (5 hours 50 minutes). These are personal estimates, not measured results.

My workflow: remove sensitive details, give clear requirements, review the answer, run checks, then use the final version.